RELUX WORKS · SWARMA

Память агентов

Ограниченная доказательная память: устройство, границы и проверка

18 октября 2026 · версия 1.2, черновик

Описание системы памяти агентов для платформы Swarma: модель данных, правила вычисления, границы полномочий, профили развёртывания и способ проверки. Точные форматы, алгоритмы и коды отказов задаёт нормативный пакет спецификации; если описание и пакет расходятся, прав пакет.

Содержание

Память агентов 3
- Назначение 3
- Принципы 3
- Модель данных 4
- Пространства памяти 5
- Версии и порядок 6
- Оценка свидетельств 7
- Представления 9
- Доступ, раскрытие и происхождение содержимого 9
- Стирание и восстановление 10
- Доставка памяти агенту 11
- Место в платформе Swarma 11
- Профили развёртывания 13
- Подписи и синхронизация 13
- Внешние агенты 14
- Предметные пакеты 14
- Нормативный пакет и проверка соответствия 15
- Состояние на октябрь 2026 года 16
- Первая рабочая версия 16
- Что не входит в первую версию 17
- Нерешённые вопросы 17
- Источники 18

Память агентов

Агенты в Swarma сменяют друг друга: сессию перезапускают, задачу передают другому исполнителю, учётку выводят из работы. При каждой такой замене теряется часть знания: почему принято решение, что от него зависит, какие доводы уже проверены и отвергнуты. Память агентов сохраняет это знание отдельно от исполнителей. Следующий агент получает основания решений, историю их пересмотра и разрешённый опыт и не повторяет уже проделанную работу.

Система называется **ограниченной доказательной памятью** (bounded evidence memory). Она хранит типизированные утверждения вместе с их источниками, обоснованиями, оценками и ревизиями. На запрос она отвечает, какие допущенные свидетельства поддерживают или опровергают точно очерченное утверждение — с чьей точки зрения и на какой момент.

Назначение

Возьмём реальный случай. В Swarma действовало правило «при выводе агента из работы его дом архивируется». Позже владелец заменил его другим: «при выводе дом удаляется», потому что доказать отсутствие секретов в содержимом дома нельзя. Обычный поиск по документам находит обе формулировки и не отличает действующую от отменённой. Память должна:

- назвать действующее решение и его точный источник;
- показать, какое решение оно заменило;
- перечислить зарегистрированные документы, которые ещё опираются на прежнее правило и требуют пересмотра;
- честно сказать, где проверка неполна.

Память отвечает на вопрос о свидетельствах, а не об истине. Она не устанавливает, что верно в мире, не решает, кто вправе действовать, и не выбирает, какое решение принять. Пустой ответ не доказывает ложность утверждения.

Знание в памяти не даёт полномочий. Запись о вчерашнем гранте не разрешает сегодняшнее действие; заметка «мы обычно пользуемся этим инструментом» не устанавливает инструмент и не меняет профиль запуска. Право на действие проверяет компонент, который это действие совершает.

Принципы

Устройство памяти держится на семи принципах; следующие разделы раскрывают их подробно.

1. **Малое ядро.** Пять видов записей и вычисляемое представление. Всё предметное — решения, профили, навыки — описывается схемами поверх ядра.
2. **Чужие журналы не копируются.** Память ведёт только историю собственных утверждений и ревизий, а события других модулей цитирует по точной ревизии их журналов.
3. **Честная оценка.** Найденное свидетельство и полнота поиска сообщаются раздельно; недоступный источник или исчерпанный бюджет не превращаются в опровержение.
4. **Данные, а не инструкции.** Агент получает память как данные с происхождением каждого элемента; текст из памяти никогда не становится инструкцией или командой.
5. **Допуск и стирание с первого дня.** Память принимает только явно допущенные классы источников, а стирание проходит по всем производным данным: выжимкам, индексам, кэшам и экспортам.
6. **Переносимость.** Смысловой контракт ядра не зависит от Swarma. Тот же корпус при тех же правилах даёт тот же ответ в любой среде; среды различаются только гарантиями аутентификации и изоляции.

7. **Автономная запись.** Агенты сами ведут память по свидетельствам, которые видят и читают. Подлинность записей держится на подписях и проверке при чтении, а не на ручном одобрении.

Модель данных

Пять видов записей

Каждая запись неизменяема: изменение оформляется новой версией или ревизией, прежние байты не переписываются.

Запись	Что содержит	Чего не означает
Утверждение (Assertion)	Типизированное высказывание о субъекте с точкой зрения, временем действия и видом высказывания	Что высказывание истинно, проверено или принято
Источник (Source reference)	Систему-владельца, объект, точную ревизию или позицию в журнале, дайджест, класс происхождения и доступность	Что источник доказывает то, о чём сообщает; сама ссылка не даёт права прочитать объект
Обоснование (Justification)	Связь утверждения с источниками или посылками: полярность (за или против), правило с версией и роль связи	Что любая зависимость является правилом вывода
Оценка (Assessment)	Результат именованной проверки с привязкой к предмету, правилу, политике и окружению	Единую отметку «проверено» на все случаи
Ревизия (Revision)	Отношение к точной прежней версии: замена, отзыв, исправление, оспаривание или изменение доступности	Что утрата основания опровергает утверждение или отменяет решение владельца

Представление (View) не хранится: это ответ, вычисленный на запрос по названному правилу. Участники и пространства знаний не образуют собственных реестров — память ссылается на личности и области политики, которые уже ведёт развёртывание.

Виды оценок различаются по смыслу. Проверка привязки к источнику подтверждает, что цитата взята из названной версии документа. Проверка подписи подтверждает происхождение, но не истинность сказанного. Результат теста относится к конкретному кандидату и окружению и не переносится на изменённый код. Смысловое ревью, в том числе выполненное LLM, остаётся суждением и не может выдать себя за детерминированную проверку вывода.

Конверт записи

Все записи несут общий конверт. Его главная задача — не смешивать того, о ком запись, того, кто за неё отвечает, и того, кто её технически внёс.

Поле	Смысл
Субъект	О ком или о чём запись
Издатель	Кто отвечает за содержание: чьё это высказывание
Отправитель	Кто технически внёс запись; устанавливается аутентифицированным приёмом, а не полем в запросе
Основание представительства	Ссылка на запись о делегировании у владельца политики, если отправитель действует от имени издателя
Точка зрения	Чью картину мира выражает запись: самого издателя, наблюдателя, команды

Поле	Смысл
Класс допуска	Самоописание, импортированная публикация, машинное наблюдение или смысловой вывод
Пространство	Область политики, к которой относится запись
Время действия	Полуоткрытый интервал в предметной области; неизвестное время хранится как неизвестное и не подменяется датой
Момент подтверждения	Когда издатель последний раз заявил или подтвердил сведения
Момент записи	Порядковый номер и время приёма в память; защищённые служебные данные

Три случая должны оставаться различимыми после любой выжимки, индексирования и экспорта:

- издатель говорит о себе;
- представитель передаёт высказывание издателя по делегированию;
- наблюдатель записывает собственное предположение о субъекте.

Если ссылка на делегирование не проходит проверку при допуске, запись отклоняется. Превращать её в собственное наблюдение отправителя нельзя.

Предметные схемы

Формат записи фиксирован ядром, а содержание утверждения подчиняется названной предметной схеме с версией. Схема определяет:

- типы полей и правило равенства значений;
- вид высказывания: факт, самоописание, предложение, запись о решении, намерение;
- кратность: одно значение на субъект и точку зрения, когда новое значение становится кандидатом на замену, или несколько сосуществующих значений;
- условия несовместимости двух утверждений: тот же субъект, пересекающиеся интервалы действия, та же точка зрения и названное отношение полей;
- горизонт свежести по умолчанию;
- ограничения раскрытия по классам полей, например «точное местонахождение никогда не публикуется».

Без правила несовместимости два утверждения не образуют противоречия: два интереса человека сосуществуют, а старое и новое правила могут относиться к разным интервалам времени. Предположение модели о том, что записи, возможно, противоречат друг другу, оформляется смысловой оценкой и детерминированным выводом не считается. Схема — это данные, а не исполняемый код; запись с неизвестной схемой или версией схемы отклоняется.

Пространства памяти

Память — не одна база, а набор пространств. Каждое пространство — git-репозиторий с файлом политики: кто владелец, кто может писать, какие классы записей принимаются, каков потолок чувствительности, сколько хранятся записи и кто разрешает конфликты.

Пространство	Чьё	Что в нём
Агента	постоянного агента: владелец, издатель и точка зрения — сам агент	память агента: что он наблюдал, какие решения принял в работе, какие гипотезы отверг

Пространство	Чьё	Что в нём
Проекта	владельца проекта	знание о проекте от всех, кто в нём работает, включая гостей
Платформы	владельцев платформы	решения и знание, общие для всех проектов

Память агента — отдельный класс записей. Это рабочее знание самого агента, а не персональные данные о людях. Агент читает и пишет сразу в несколько пространств в пределах своих грантов.

Центральной базы нет. Представление строится по всем пространствам, которые вызывающий вправе читать, с фильтрами по издателю и точке зрения. Каталог пространств в пространстве платформы связывает личности и проекты с адресами их пространств и правами доступа. По нему агент находит память другого агента, если владелец дал к ней доступ.

Знание переходит в общую память новой записью в пространстве проекта или платформы со ссылкой на исходную запись агента. Оригинал остаётся на месте, связь прослеживается, издателем по-прежнему остаётся агент. Запись «платформа приняла решение» требует квитанции владельца.

Если агент записывает то, что произошло в сессии, запись содержит сам фрагмент — утверждение или допущенную нечувствительную выдержку, потому что сессию могут удалить. Ссылка на источник указывает среду исполнения, идентификатор сессии, позицию сообщения и дайджест его содержимого. Позиция — идентификатор сообщения, если среда его даёт, иначе индекс в сохранённой последовательности сообщений. После удаления сессии запись остаётся, а источник помечается недоступным.

Версии и порядок

Право на ревизию

Записи одного издателя об одном субъекте по одной схеме и с одной точки зрения образуют последовательность версий. Заменить, исправить или отозвать запись в последовательности может только её издатель или его допущенный представитель. Любой другой участник может оспорить запись или записать собственную оценку в своей последовательности.

Поэтому поздняя заметка агента «дом архивируется» не заменяет решение владельца об удалении дома. Она остаётся отдельным утверждением наблюдателя или оспариванием.

Владелец проекции может исключить чужую запись из своего представления — например, убрать аттестацию организации из своей публичной карточки. Это выбор правила проекции, а не ревизия: запись организации и её последовательность не меняются.

Порядок решений

Действующую версию решения определяет не память, а упорядоченный журнал того, кто решал: порядок коммитов реестра решений после ревью или ревизия доски задач. Момент, когда память узнала о решении, на порядок не влияет: решение, полученное с опозданием, занимает своё место в журнале решающего.

Порядок решений издателя задают только его собственные события. Пусть V_H — события издателя H, а E_H — рёбра «событие → его предшественник» между событиями из V_H. Событие a предшествует событию b в порядке H, если из b в a ведёт непустой путь по рёбрам E_H. При этом:

- промежуточные события издателя без привязанного утверждения тоже являются звеньями пути;
- достаточно одного такого пути, и соседние ветви через события других издателей его не отменяют;

- ребро через событие другого издателя порядка для H не даёт, а выбросить такое событие, соединив его соседей напрямую, нельзя: так появилось бы свидетельство, которого нет;
- изменение только той части графа, что принадлежит другим издателям, порядок H не меняет.

Графы с событиями разных издателей допустимы. Если между двумя кандидатами нет пути из событий издателя, порядок не установлен: представление возвращает `ambiguous_current` с обоими кандидатами и ни одного не объявляет устаревшим. Тем же отношением проверяется, что авторитетная замена строго следует за заменяемой версией; запись, которая этому не соответствует, отклоняется с кодом `authority_receipt_required`.

Конкурентная запись и три вида конфликтов

Ревизия указывает ожидаемую версию. Если последовательность уже ушла вперёд, запись отклоняется с кодом `version_conflict` и ничего не фиксируется; правило «побеждает последний записавший» не применяется.

Слово «конфликт» в памяти обозначает три разных результата, и API всегда их различает:

Результат	Где возникает	Что означает
`version_conflict`	Отказ операции записи	Запись основана на версии, которая уже не последняя; никаких сведений о субъекте не создаётся
`ambiguous_current`	Ответ представления	В журнале решающего нет порядка между двумя текущими кандидатами
Конфликт свидетельств	Результат оценки	Найдены допустимые основания и за утверждение, и против; обе стороны сохраняются

Эскалация конфликтов

Если конфликт касается решения, от которого зависит текущая работа, шаг, построенный на этом решении, останавливается, и конфликт передаётся тем, кто по политике пространства отвечает за его разрешение. Они принимают новое решение; оно заменяет конфликтующие по обычным правилам порядка, и работа продолжается. Сама память эскалацию не поднимает и никого не будит: это делает агент или оркестратор, который встретил конфликт, через штатный путь эскалации платформы. Конфликты, от которых работа не зависит, остаются видимыми и остальному не мешают.

Оценка свидетельств

Результат по полярности

Поддержка и опровержение оцениваются независимо. Для каждой полярности ответ содержит:

Поле	Значения
Свидетель	`present` — найдено допустимое основание; `not_found` — не найдено
Оценка	`complete` — вычисление по зарегистрированному разрешённому графу завершено; `incomplete` — прервано
Покрытие корпуса	Насколько записи представляют исходный материал: неизвестно или заявлено тем, кто их регистрировал
Качество посылок	Есть путь без конфликтных посылок; поддержка найдена только через конфликтные посылки; проверено не полностью

Поле	Значения
Состояние источника	Источник доступен сейчас; сохранилась только прежняя оценка; есть самостоятельная достаточная аттестация
Свежесть	Сведения свежие, устарели или свежесть к ним неприменима
Квалификация	Каким видом проверки подтверждён свидетель

Сочетания читаются так:

- `not_found` при `incomplete` означает «неизвестно», а не завершённый вердикт;
- `present` при `incomplete` доказывает существование найденного основания, но не позволяет сказать, что возражений нет;
- свидетели обеих полярностей устанавливают конфликт, даже если перебор не закончен;
- недоступный источник, отсутствующие байты, незавершённый приём и исчерпанный бюджет никогда не становятся опровержением.

Полнота и её пределы

Полнота оценки относится к зарегистрированному разрешённому графу на выбранный момент — не к миру и не к исходным документам. Если в реестре три основания и все три проверены, оценка полна; но это не доказывает, что при подготовке реестра не пропустили четвёртое основание в документе. Поэтому покрытие корпуса сообщается отдельным полем и может оставаться неизвестным при полной оценке.

Поиск кандидатов и проверка свидетельств — разные операции с разными обещаниями: выдача «лучших десяти совпадений» не даёт права объявить оценку полной. Пересмотр при изменении основания охватывает зарегистрированные зависимости, но не все смысловые следствия текста на естественном языке.

Усечённый показ объяснений и усечённое вычисление тоже различаются. Объяснение может быть сокращено при полной оценке; прерванное вычисление обязано вернуть `incomplete`.

Правила вывода

Первой версии достаточно явных источников, документальных оснований, замен и связей «пересмотреть при изменении». Документ, который зарегистрировал допущение об архивировании дома, возвращается как требующий проверки. Память не объявляет его неверным: пересмотру может подлежать только часть документа. Прозаическое объяснение само по себе не даёт права вычислять логические следствия.

Следующий срез добавляет конечные ациклические правила над знаковыми высказываниями: внутри набора посылок — «и», между альтернативными правилами — «или». Если C поддерживается через A и B либо через D и E, отзыв B выключает только первый путь, и C остаётся поддержанным через D и E. Правила односторонние: когда отпали все пути, отрицание C из этого не следует. Отрицания по неудаче, контрапозиции и произвольного исполняемого кода в правилах нет.

Полная система сопровождения допущений (ATMS) не используется. Для семейства $C = И(A_i \text{ или } B_i)$ явный перечень окружений допущений растёт как 2^n, а граф зависимостей с обратным индексом имеет $O(n)$ вершин и отвечает на вопрос о существовании свидетеля одним проходом. ATMS понадобится, только когда появится измеренный сценарий с запросами по гипотетическим окружениям.

Конфликтные посылки

Каналы «за» и «против» независимы. Если у A есть основания и за, и против, а правило $A^+ \to B^+$ допущено, B получает поддержку с пометкой о конфликтной посылке, но не автоматическое опровержение. Компактный ответ обязан сохранить эту пометку, чтобы потребитель видел: поддержка найдена

только через конфликтную посылку. Можно ли действовать на таком основании, решает политика потребителя.

Представления

Правила построения

Представление вычисляется по названному правилу с версией. Правило задаёт, какие издатели и точки зрения учитываются, на какой момент строится ответ, как разрешаются последовательности версий и какая действует политика свежести. Ядро определяет три семейства правил:

Семейство	Что возвращает	Пример
Текущее от издателя	Действующие высказывания издателя о субъекте	Самоописание участника; действующие решения владельца
Модель наблюдателя	Собственные утверждения и оценки одного наблюдателя о субъекте	Что агент-координатор знает о работе ревьюера
Проекция для аудитории	Часть текущего от издателя, разрешённая для названной аудитории и цели	Карточка участника для поиска соавторов

Ядро никогда не выбирает «настоящий профиль» или «действующее решение» по числу свидетельств или уверенности модели.

Свежесть

Различаются три момента:

- когда сведения действуют в предметной области;
- когда издатель последний раз их заявил или подтвердил;
- до какого момента потребитель готов пользоваться ими без обновления.

После этого горизонта сведения помечаются `stale`: их нужно обновить, но они не становятся ложными и не считаются опровержением. Для намерений, доступности и местонахождения горизонт обычно задан; для архитектурных решений он обычно не нужен.

Снимки

Зафиксированное представление, в том числе снимок памяти в начале сессии, неизменяемо как исторический артефакт своего момента: его байты не меняются, меняется применимость. Различаются два случая.

Знание обновилось. Решение заменено, появилась новая оценка, истёк горизонт свежести. Снимок остаётся верной историей, а текущее состояние агент получает новым запросом. Элемент снимка всегда показывается вместе с его моментом и никогда — как «сейчас».

Использование или раскрытие отозвано. Компонент, который выполняет следующую доставку или внешний эффект, применяет текущую политику: отказывает в операции или пересобирает допущенный контекст. Память только сообщает об изменении и сама агента не перезапускает. Кэш промпта не продлевает разрешений, а байты, уже переданные модели, вернуть нельзя.

Доступ, раскрытие и происхождение содержимого

Разрешения

Чтение, использование в задаче, хранение и дальнейшая передача требуют раздельных разрешений. Право читать не даёт права сохранить байты или отправить их провайдеру LLM или эмбеддингов. Представление, допущенное для одной задачи, не становится автоматически допустимым для другой

задачи или для восстановленного исполнения: перед разрешением источника и перед новой доставкой права проверяются заново.

Невлияние скрытых данных

Скрытые записи не влияют на то, что видит получатель: на раскрытые статусы, счётчики, форму графа, ранжирование, курсоры, покрытие и уведомления об изменениях. Права проверяются до вычисления, а не только перед показом текста. Логический бюджет запроса учитывает только разрешённую работу, иначе большое число скрытых записей выдало бы себя переходом ответа в `incomplete`.

Публичные ручки, курсоры, квитанции и классы ошибок зависят только от допущенной проекции. Один из способов этого добиться — непрозрачные ручки фиксированной длины, случайные на каждый запрос, с защищённой привязкой для постраничного чтения. Обещание касается смысла и метаданных ответа; постоянного времени ответа и защиты от анализа трафика оно не даёт.

Происхождение содержимого

Логическая поддержка и происхождение содержимого — разные связи: выжимка может пересказать частную запись, не используя её как посылку вывода. Поэтому каждая производная запись — выдержка, пересказ, эмбеддинг, запись индекса, кэш представления, результат сопоставления — несёт перечень своих входов и наследует ограничения раскрытия всех этих входов. Метка «публично» ограничений не снимает: пересказ приватной записи остаётся приватным.

Перечень входов составляет интеграция, а не агент; список цитат, приложенный агентом, полным происхождением не является. Для детерминированного преобразования учитываются прочитанные входы, для вызова модели — всё, что адаптер передал в этот вызов: контекст, результаты инструментов, представления памяти. Если происхождение установить нельзя, запись наследует самые строгие ограничения из переданных входов и по умолчанию не публикуется. Обещание ограничено тем, что отслеживает профиль интеграции; о знаниях модели вне переданного контекста память ничего не утверждает.

Снятие ограничений

Раскрыть результат вычисления над закрытыми входами может только решение владельца политики каждого ограниченного входа. Например, разрешение опубликовать город, вычисленный из приватных координат, не разрешает публиковать сами координаты. Если входы принадлежат нескольким владельцам, решение одного снимает только его ограничение.

Решение фиксируется оценкой с привязкой к входам, преобразованию, выходной схеме, аудитории и цели. Такая оценка — свидетельство решения, а не полномочие: перед каждым новым раскрытием проверяется, что решение всё ещё применимо. Мнение модели о безопасности пересказа остаётся смысловой оценкой и может только предложить такое решение. Получатель проверяет заявление издателя, но не скрытый вывод.

Идентификаторы субъектов

Внутренний идентификатор субъекта никогда не служит публичным: для каждого пространства и каждой проекции субъект получает свою ручку. Связь между двумя представлениями одного субъекта, например профессиональной и личной карточками, сама является раскрываемыми сведениями и требует отдельного разрешения; общий идентификатор не должен выдавать её побочно.

Стирание и восстановление

В защищённой службе логическая история отделена от хранения содержимого. Чувствительные байты утверждений и выдержек лежат в стираемых объектах, а не в вечном журнале аудита. Стирание идёт по линии происхождения содержимого, а не по графу рассуждений, и охватывает выдержки, пересказы, эмбеддинги, индексы, кэши, контролируемые экспорты, очереди и диагностику.

Стирание выполняется в три шага:

1. Ограждаются чтение и фиксация производных для затронутого поколения данных. Запись от позднего извлекателя со старым поколением отклоняется, как и запись, выведенная из стёртых входов.
2. Каждое контролируемое хранилище очищается и выдаёт квитанцию.
3. Исход сообщается одним из четырёх состояний: доступ закрыт, очистка ожидает, очищено в названной границе, недоступно.

Восстановление из старой копии остаётся закрытым, пока внешний свидетель вне восстановленного домена отказа не подтвердит текущую полную проверку всех действующих обязательств стирания. Для этого недостаточно ни самого существования внешнего подписанного свидетеля, ни свидетеля, покрывающего только прежние обязательства, ни списка стираний из восстановленной копии. Если затронутое хранилище недоступно, восстановление отклоняется целиком.

У стирания есть пределы. Удаление у независимого владельца не заявляется без его пути удаления и квитанции. Байты, уже доставленные внешней модели, отозвать нельзя. Логически неизменяемая история не обещает физически хранить любые байты вечно: после законного стирания часть обоснований становится непредъявимой, и представление честно об этом сообщает.

Доставка памяти агенту

Агент получает три раздельных входа: авторитетную текущую задачу и цель, пакет продолжения работы от хоста сессий и необязательные данные памяти. В первой версии память приходит только явным результатом инструмента: агент сам её запрашивает. Автоматического добавления памяти при запуске нет, и формат пакета передачи между сессиями для этого не расширяется.

Каждый элемент несёт автора, авторитетный источник и пометку `data`; они сохраняются при пересказе, раскрытии, кэшировании, повторном импорте и передаче смены. Подпись службы удостоверяет конверт, но не повышает полномочия вложенного текста. Императив, спрятанный в подписанном источнике, на всём пути остаётся данными.

Позже появится снимок памяти в начале сессии, и его канал определён заранее: отдельный блок данных с происхождением каждого элемента. Память никогда не попадает в оверлеи инструкций Swarm Curator, в `AGENTS.md`, каталоги скиллов, хуки, конфигурацию MCP или машинный базовый контекст. Вес оверлея только упорядочивает главы инструкций и не превращает инструкцию в данные, поэтому веса инструкций у памяти нет.

Пометка `data` сама по себе не мешает модели последовать вредоносному тексту. Гарантия памяти в другом: внешние проверки полномочий не меняются, что бы ни было написано в данных. Устойчивость модели к инъекциям измеряется отдельно.

Место в платформе Swarma

Память — необязательный клиент базовых модулей платформы. Она ссылается на их записи и ничего за них не решает.

Сведения	Авторитетный владелец	Что видит память
Поколение учётки, вывод из работы, применённые границы машины	Менеджер пользователей	Версионированное свидетельство; старая запись не является текущим разрешением
Личности, их ключи и гранты представительства	Реестр владельца	Ссылки на ячейки личностей и гранты, проверенные по снимку реестра; второго реестра делегирования нет

Сведения	Авторитетный владелец	Что видит память
Учётные данные, гранты на доступ, отзывы, аренды	Брокер учётных данных и владельцы политики	Ссылки без секретов и без второго реестра прав
Допуск, размещение, исполнение, уборка	Диспетчер	Ссылку на каноническую запись исполнения, а не на отчёт агента о нём
Журналы сессий, квитанции доставки, ограждения ввода	Хост сессий	Допущенный экспорт только для чтения, без доступа ко вводу сессии
Цели, жизненный цикл задач, принятый кандидат	Доска задач	Зафиксированные ревизии; изменения и приёмка остаются у доски
Обязательства, сроки, эскалация	Swarm Keeper	Ссылки на свидетельства, без второго реестра обязательств
События комнат, квитанции транспорта	Мост сообщений	Свидетельство об отправителе и доставке в пределах гарантий транспорта
Профили запуска, скиллы, MCP, каналы инструкций	Swarm Curator	Сведения о конфигурации без права её менять
Утверждения и ревизии памяти	Сама память: подписанные записи в git-пространствах или единственный писатель защищённой службы	Только собственная история знаний
Индексы и кэши представлений	Адаптер или служба, которые их строят	Перестраиваемые проекции, не источник истины

Личности и делегирование

Участники, которых ядро называет издателем, субъектом и отправителем, в Swarma — ячейки личностей. Ячейка хранит текущие ключи, хэш следующих ключей и ссылку на родителя-владельца. Состояние ячеек ведёт реестр владельца — единственный авторитетный писатель, с журналом в git для аудита и подписанными снимками ограниченной давности.

- **Отправитель** определяется по аутентифицированной ячейке вызывающего в текущем снимке реестра.
- **Основание представительства** — грант в реестре. При допуске записи адаптер проверяет по снимку издателя, отправителя, область, срок и то, что ни грант, ни ячейки не израсходованы. Недоступный реестр никогда не считается выданным грантом: запись отклоняется.
- **Смена ключей** расходует ячейку следующими ключами и действует с момента публикации. Записи, принятые раньше, сохраняют свою личность и историю; их подписи проверяются по ключам, действовавшим в момент приёма.
- **Отзыв** — расходование гранта или перевод ячейки в терминальное состояние. Новые записи от её имени отклоняются, прежние остаются историей. Текущее от отозванного издателя помечается устаревшим; преемника память сама не назначает.

Ядро при этом не меняется: привязка описана в контракте адаптера.

Чего память не делает

У памяти нет операций «одобрить», «принять задачу», «поставить цель», «выдать грант», «запустить» и «разбудить». Изменение основания может стать поводом для ревью, но открыть работу и принять её результат должен уполномоченный клиент. Уведомления об изменениях — это опрос советующих изменений в истории памяти, без пробуждения сессий и постановки целей.

Остановка, парковка, отзыв прав, восстановление и приёмка никогда не ждут памяти, вывода или консолидации. Если конкретному действию нужно свежее свидетельство, а память недоступна, отказывает это действие, а не узел.

Профили развёртывания

Git-пространства

Первый профиль — пространства в git, автономная запись и проверка при чтении. Демона и служебной учётки нет.

- **Запись.** Агент пишет сам. Команда записи проверяет запись локально — схему, право на ревизию и ожидаемую версию, — подписывает её и отправляет в репозиторий. Если кто-то успел раньше, она забирает новое состояние, проверяет ожидаемую версию заново и повторяет попытку. Человек пишет тем же путём под своей личностью или просит агента. Ревью и одобрения на пути записи нет.
- **Проверка при чтении.** Каждый читатель сам проверяет подпись, схему, политику пространства и право на ревизию. Запись, не прошедшая проверку, показывается отдельно и в ответах не учитывается. Читатель помнит последние проверенные состояния пространств, поэтому переписывание истории заметно.
- **Допуск.** Принимаются только названные классы источников. Потолок — нечувствительное содержимое: из истории git ничего не стирается. Рабочее правило: если содержимое не положили бы в README репозитория, ему не место в пространстве. Автоматический поиск секретов при записи отсекает очевидное, но отсутствия секретов не доказывает.
- **Внешние факты** атрибутируются своему источнику и классу допуска и за знание агента не выдаются.
- **Извлечение.** Сырые журналы сессий не принимаются: агент записывает то, что сам счёл значимым. Детерминированный дистиллятор журналов и извлечение моделью рассматриваются, только когда польза рабочей версии будет подтверждена.

Корректная схема, хэш, подпись или фильтр LLM не доказывают отсутствия секретов. Поэтому в git-пространства не попадают личные данные людей, сырые разговоры, учётные данные и произвольный вывод инструментов.

Защищённая служба

Второй профиль нужен для того, чего git дать не может: чувствительных данных, гарантированного стирания и проверки записи до её попадания в хранилище. Его ввод привязан к появлению в платформе отдельного пользователя ОС и посредника.

Служба — один непривилегированный процесс с единственным писателем и тем же смысловым контрактом. До развёртывания у неё должны быть:

- закрытое состояние и единственный API записи;
- опубликованные права на сокет и проверка UID собеседника вместе с текущим поколением его учётки;
- удостоверенная личность сервера и явно допущенные роли клиентов;
- собственная строка выхода в сеть, которой владеет установщик;
- проверка готовности.

Имя служебной учётки само по себе ни одной из этих гарантий не даёт. Прохождение смысловых проверок не доказывает изоляцию службы: её квалификация — отдельная работа.

Подписи и синхронизация

Подписи

Каждая запись подписана отправителем. Подпись покрывает полные канонические байты записи с меткой назначения, поэтому подпись записи памяти нельзя выдать за подпись сообщения, квитанции или аттестации. Непрозрачный хэш на подпись не отправляется никогда: тот, кто подписывает

от имени человека или другого владельца, получает подписываемое целиком и проверяет его. Подпись коммита git остаётся для транспорта и истории, а доверие к записи держится на подписи самой записи. Поэтому запись проверяема и вне git: при экспорте, переносе и якорении.

Постоянный агент Swarma закрытого ключа не держит. Запись подписывает ключник по запросу агента: он получает её целиком и проверяет, что издатель — тот, кто просит. Пока ключника нет, агент сам генерирует ключ подписи и подписывает им свои записи; это временное исключение. Читатель проверяет подпись по ключу, удостоверенному корнем владельца, а затем по ячейке личности.

Синхронизация

Движок даёт примитивы, а скилл предписывает агенту, когда ими пользоваться:

- `memory sync` забирает все доступные пространства, проверяет новые записи и сдвигает курсор по каждому пространству;
- в начале задачи — синхронизация и запрос по теме задачи;
- перед каждой записью — синхронизация, затем запись и отправка, при гонке — повтор;
- на контрольных точках и перед сдачей работы — изменения с последнего курсора;
- в конце задачи — запись того, что агент узнал.

Уведомлений об изменениях память сама не рассылает.

Внешние агенты

Агент другого владельца, который работает у нас, в первой версии своей памяти не имеет: он эфемерен. Путь для следующих версий определён.

- При допуске в проект платформа проверяет его внешнюю личность: публичные ключи и цепочку удостоверений до корня его владельца. Закрытых ключей внешнего агента у нас нет, нашим ключником он не пользуется; на нашей стороне у него только контекст полномочий.
- Пишет он в пространство проекта как гость; его записи привязаны к идентификатору его ячейки личности.
- Подписывает он своим ключом на своей стороне — в мосте донора, а не инструментом модели. Мост получает подписываемое целиком и проверяет его. Первый путь — записи памяти внутри подписанного экспорта результата; отдельное сообщение моста для записи по ходу задачи — возможное расширение.
- Вернувшись и доказав, что он тот же, агент снова видит свои прежние записи в проекте: память следует за личностью, а не за ключом. Остальной доступ определяет политика.
- Знание о нашем проекте остаётся у нас. Отдать его владельцу агента можно только отдельным экспортом по решению владельца проекта.

Предметные пакеты

Ядро ничего не знает о решениях, людях и профилях. Предметный смысл задают пакеты схем и правил проекции поверх ядра; пакет не расширяет полномочий памяти.

Архитектурные решения

Пакет описывает решения, правила и зарегистрированные допущения. Решения «архивировать дом» и «удалять дом» — две версии одной последовательности владельца. Запрос текущего от издателя возвращает действующее решение с квитанцией, а прежняя версия остаётся прослеживаемой. Документы, которые зарегистрировали прежнее правило как допущение, связаны с ним связями «пересмотреть при изменении» и возвращаются как требующие проверки.

Профили людей и автономных агентов

Пакет описывает типизированные представления об участниках: интересы, компетенции, местонахождение, доступность, намерения, связи с внешними аккаунтами, статус отношений и готовность к знакомствам. Он проверяет общность ядра и пока работает только на синтетическом корпусе; реальные личные профили требуют защищённой службы.

Схемы пакета разводят понятия, которые легко спутать:

- интерес и компетенцию;
- заявленную компетенцию и опыт конкретных работ;
- место проживания, текущее местонахождение и район работы;
- заявленную ссылку на аккаунт и проверенный контроль над ним;
- намерение и обязательство;
- статус отношений и готовность к романтическим знакомствам, которые не выводятся друг из друга.

Догадка агента об интересе владельца — запись модели наблюдателя; самоописание владельца агент передаёт только по делегированию. История успешных ревью хранится вместе с конфигурацией исполнения, но вывод «этому агенту можно доверить следующую работу» остаётся решением потребителя.

Публикация распределена между ролями. Память строит проекцию для аудитории; издатель выпускает из неё карточку со своим идентификатором и ревизией; каталог хранит и индексирует публикацию; память другого участника импортирует её как внешний источник. Одна самодекларация, прошедшая через три каталога, остаётся одним корневым свидетельством. Без проверки текущей версии у издателя импортёр знает только последнее полученное состояние.

Четыре действия с опубликованными сведениями независимы:

Действие	Результат	Чего не означает
Изменить утверждение	Новая версия в последовательности издателя	Что прежняя версия была ложной
Отозвать дальнейшее раскрытие	Новые представления и отправки для этой аудитории отклоняются	Отзыва уже отправленных копий
Снять публикацию	Издатель отзывает карточку	Стирания хранимого содержимого
Стереть содержимое	Стирание по линии происхождения в контролируемой границе	Удаления у независимых получателей

Поиск собеседников, правила обращений и знакомства — сервисы над ядром. Для них заданы умолчания безопасности: точного местонахождения нет ни в одной публичной карточке; романтические знакомства включаются только явно и только для взрослых; контакты раскрываются после взаимного согласия; публичная связь «мы партнёры» показывается только после подтверждения обеих сторон.

Нормативный пакет и проверка соответствия

Точные правила ядра собраны в нормативном пакете спецификации, черновик 0.2.7. В него входят:

- нормативный текст ядра — 52 пункта с алгоритмами, порядком операций и кодами отказов, без терминов Swarma;
- 14 закрытых схем JSON Schema 2020-12;
- предметные пакеты архитектурных решений и синтетических профилей;
- контракт адаптера Swarma;
- 74 вектора соответствия: 66 смысловых и 8 для защищённого развёртывания;

- структурная проверка на стандартной библиотеке Python.

Каждый вектор задаёт полный начальный снимок, операции, запрос, точный ожидаемый ответ или отказ и запрещённые исходы. Смысловые векторы и векторы развёртывания отчитываются раздельно: прохождение первых не означает прохождения вторых.

Переносимость проверяется на одинаковом наборе входов: корпус, точные версии схем и правил проекции, точка зрения, разрешённая проекция, момент оценки и логический бюджет. При одинаковых входах ответы должны совпадать по смыслу; случайные непрозрачные ручки сравниваются после оговорённой нормализации.

Однозначность пакета проверена двумя независимыми вычислителями. Первый написан по сводному контракту ядра без доступа к пакету и затем прогнан на векторах пакета без их правки; второй написан только по пакету. Каждое расхождение между ними и ожидаемыми ответами разбиралось как ошибка пакета, ошибка реализации или неоднозначность контракта; все расхождения закрыты.

Проверка	Результат на октябрь 2026 года
Смысловые векторы	Оба вычислителя дают ожидаемые ответы на всех 66
Дополнительные состязательные случаи	Оба вычислителя проходят все 68
Векторы защищённого развёртывания	Не исполняются: для них нужна настоящая служба, и синтетические трассы её не заменяют

Этот результат означает, что две независимо написанные реализации согласованы с эталонами на зафиксированном наборе случаев. Он не доказывает корректность на всех допустимых входах, не квалифицирует защищённую службу и ничего не говорит о пользе памяти в реальной работе.

Состояние на октябрь 2026 года

Часть	Состояние
Принципы, границы полномочий, модель данных, семантика оценки и раскрытия, порядок решений, привязка личностей и делегирования	Решено; не реализовано
Нормативный пакет	Решено; черновик спецификации 0.2.7 проверен двумя вычислителями
Пространства, автономная запись, подписи, синхронизация, эскалация конфликтов	Решено; не реализовано
Первая рабочая версия	Решено: Go, один репозиторий; архитектура готовится
Защищённая служба	Решено; не построена и не квалифицирована
Память внешних агентов	В первой версии нет; путь определён
Реальные личные данные	Не допускаются до появления защищённой службы
Польза и стоимость в реальной работе	Не измерены

Первая рабочая версия

Первая рабочая версия — git-пространства с автономной записью, подписями и проверкой при чтении. Её код живёт в одном репозитории `swarma-memory` на Go: модуль ядра, CLI `memory` и скилл, через который Swarm Curator ставит CLI агентам. Ядро на Go обязано пройти все смысловые векторы нормативного пакета. Архитектура рабочей версии описывается отдельным документом.

Польза рабочей версии оценивается на реальной работе агентов. Чтобы отделить пользу вычислителя от пользы аккуратно подготовленных записей, часть вопросов сравнивается с обычным поиском и с

типизированными версионированными документами при одинаковых источниках, правах и бюджетах.

Метрика	Что показывает
Правильное текущее решение с точным источником	Основную точность
Доля устаревших ответов	Как часто отменённое выдаётся за действующее
Точность свидетельств	Подтверждают ли источники ответ
Пропущенные зарегистрированные зависимости	Полноту пересмотра
Ложные призывы к пересмотру	Шум
Время ответа и повторное чтение источников	Стоимость получения ответа
Стоимость поддержания корпуса	Время людей и агентов на подготовку, ревью и обновление записей
Задержка актуализации	Время от принятия решения до момента, когда агенты перестают получать прежнее как действующее

Отказы и исчерпание бюджета остаются в знаменателе. Рабочая версия полезна, если новый агент продолжает работу, не возвращает отменённое решение, правильно указывает основания и не объявляет непроверенные гарантии установленными — при приемлемой стоимости ведения памяти. Автоматическое извлечение, более богатый вывод или другой графовый движок оправданы, только если улучшают этот результат, не ослабляя границ.

Что не входит в первую версию

- полная система сопровождения допущений, общий Datalog и доказательство теорем;
- автоматический приём транскриптов и извлечение моделью из журналов;
- личные предпочтения и реальные личные данные;
- активация процедурной памяти и автоматическое превращение опыта в скиллы;
- расписание ночной консолидации и автоматические ревью;
- репутация агентов как основание допуска;
- собственная память внешних агентов;
- межвладельческая федерация с конвертами знаний, консенсус и якорение в Swarm Chain;
- распределённая служба памяти на нескольких машинах: совместные пространства синхронизируются через git;
- готовые графовые хранилища вроде Graphiti — их можно подключить позже как заменяемую проекцию, после проверки удаления на точных версиях.

Репетиция процесса без реальных внешних эффектов образует отдельный контекст свидетельств. Смоделированная приёмка или результат теста не удовлетворяют требованию реального исполнения, а выжимка репетиции подтверждает только саму репетицию.

Нерешённые вопросы

- Где размещается пространство постоянного агента и что с ним происходит при выводе учётки. Дом агента при выводе удаляется и не архивируется, поэтому память не может жить только в нём.
- Остаточный риск секретов в памяти, которая переживает дом: строгие схемы и поиск секретов отсутствия секретов не доказывают.
- Кто регистрирует временный ключ агента как принадлежащий ему и как заменить потерянный ключ, не разорвав историю.

- Кто по каким темам разрешает конфликты в общих пространствах и через какой путь идёт эскалация.
- Формат и владелец каталога пространств.
- Какие классы источников и как долго должны оставаться предъявимыми.
- Какие реальные классы источников допускаются и с какими сроками и границами хранения.
- Производственные пределы ресурсов и механизм восстановления защищённой службы.
- Какие грубые сведения о доступности службы можно раскрывать заранее.
- Квалификация защищённого развёртывания.
- Когда ядро выделяется в самостоятельный открытый выпуск; ожидается, что при появлении второго реального потребителя.

Источники

- Нормативный пакет спецификации, черновик 0.2.7: `memory/spec/` в репозитории `swarma-architecture-review` — нормативный текст ядра `kernel/semantics.md`, схемы, векторы, предметные пакеты, контракт адаптера.
- Сводный контракт ядра: `memory/designs/kernel.md`.
- Решения по памяти: `memory/DECISIONS.md`; открытые вопросы и риски: `memory/OPEN-QUESTIONS.md`.
- Отчёт о проверке соответствия: `memory/conformance/`.
- Книга Swarma: главы о сети агента, хосте сессий, мосте сообщений и доске задач.